
RentalHub

System Architecture and Low-Level Design

India-first equipment rental marketplace with booking, delivery, inspection, returns, disputes and maintenance

Stack	React, Vite, Tailwind, React Router, Axios, React Query, Socket.IO / Node.js, Express, MongoDB, Mongoose, JWT, bcrypt, Multer
Roles	CUSTOMER, OWNER, TRANSPORTER, INSPECTOR, ADMIN
Phases	1 Auth + Marketplace, 2 Booking + Rental, 3 Delivery + Inspection, 4 Returns + Damage + Maintenance
Locale	INR, English default, Hindi, Tamil; Indian phone, address and date conventions
Theme	White + Red (#D32F2F, #B71C1C, #FFFFFF, #F7F7F7, #1F1F1F, #E5E7EB)

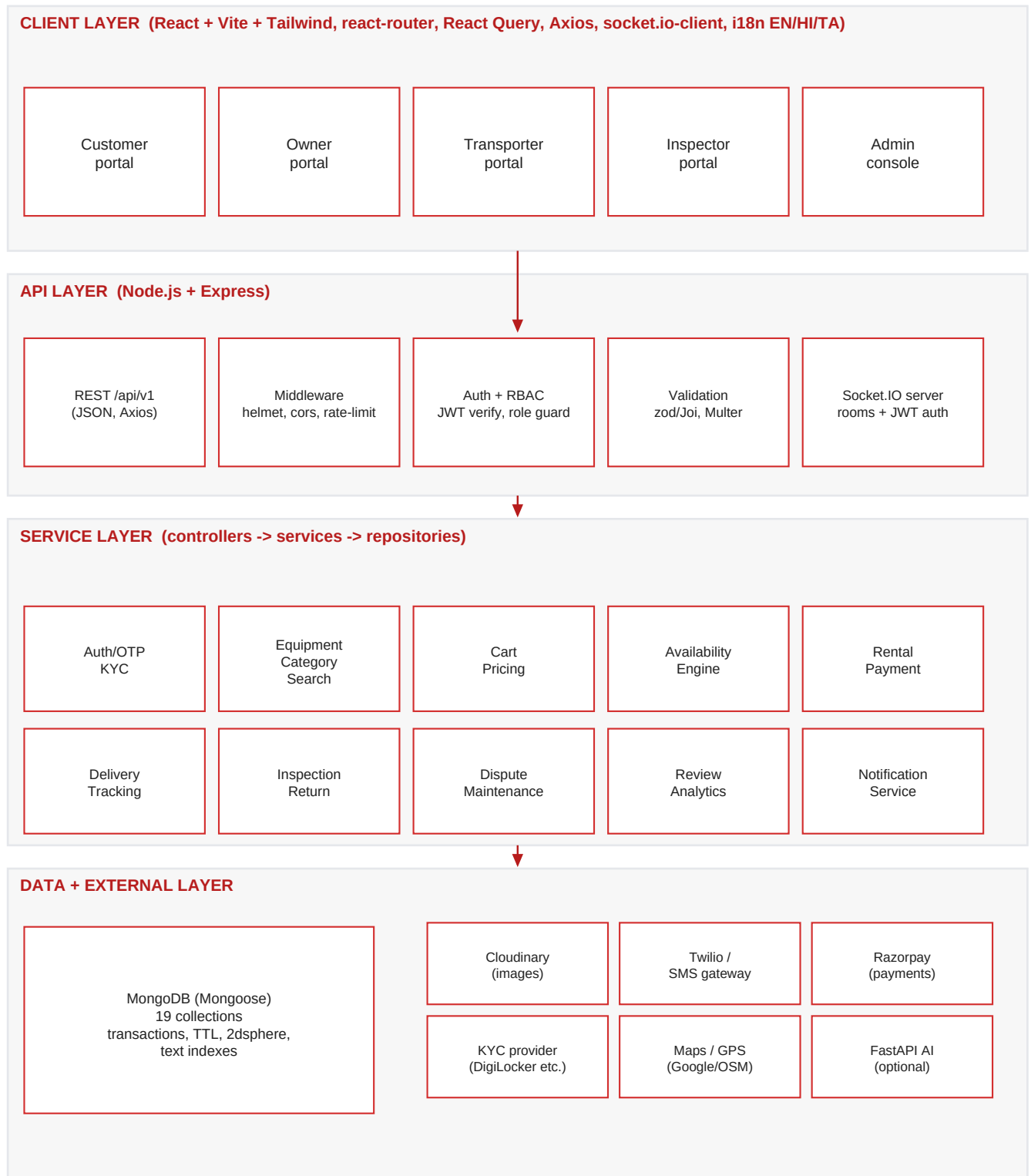
1. Core Features

Phase	Feature area	Key capabilities
1	Auth and identity	Register, login, logout, JWT access + refresh, bcrypt hashing, forgot/reset/change password, real OTP (SMS via Twilio), role-based access, four role profiles
1	KYC	Provider-based verification; store only verification reference, status, last-4 and masked ID, never raw Aadhaar
1	Owner inventory	Equipment CRUD, image uploads (Multer to Cloudinary), dynamic categories, quantity, rates, specs
1	Marketplace	Dynamic listing from MongoDB, search, filter, sort, pagination, English/Hindi/Tamil multilingual search, detail page, rental date selection, database-backed cart
2	Availability and pricing	Quantity + date-overlap engine, server-side price calculation, checkout, payment-service abstraction
2	Booking	Rental creation, owner accept/reject, cancellation workflow, 11-state rental lifecycle, customer and owner dashboards
2	Notifications	Persisted notifications plus Socket.IO real-time push
3	Delivery	Transporter assignment and dashboard, delivery lifecycle, real GPS tracking over Socket.IO, delivery failure workflow
3	Inspection at delivery	Condition checklist, photos/evidence, meter readings, damage/missing-component records, customer handover acknowledgement
4	Returns	Return workflow, return transporter, return inspection, delivery-vs-return comparison, Equipment Passport
4	Disputes	Evidence-based dispute with OPEN, UNDER_REVIEW, ACTION_REQUIRED, RESOLVED, CLOSED; system never assigns blame automatically
4	Maintenance and insight	Maintenance records that block availability, prediction architecture, reviews after completion, owner reliability, utilization and smart-pricing computed from real data

Design principle: every business value (products, bookings, payments, OTPs, GPS points, inspections, notifications, analytics) originates from MongoDB, the API, or a real external provider. Where a provider is not configured, the feature returns a clear 'not configured' error instead of fake data.

2. System Architecture

2.1 Layered architecture



The application is a modular monolith: one Express deployable with strictly separated feature modules. Modules communicate through service interfaces, so any module (for example Tracking or Analytics) can be extracted into its own service later without rewriting callers.

2.2 Technology responsibilities

Concern	Choice	Responsibility
UI	React + Vite + Tailwind	Role-based portals, responsive white/red design system
Routing	React Router v6	Protected routes by role, lazy-loaded route bundles
Server state	React Query	Caching, pagination, optimistic cart updates, invalidation on socket events
HTTP	Axios	Single instance, JWT interceptor, silent refresh on 401, Accept-Language header
Realtime	Socket.IO	Authenticated rooms per user, rental and delivery
i18n	i18next (en, hi, ta)	UI strings; INR via Intl.NumberFormat('en-IN'); dates as DD/MM/YYYY
API	Express + zod	Validation, error envelope, versioned /api/v1
Persistence	MongoDB + Mongoose	Documents, transactions (replica set), TTL, text and 2dsphere indexes
Files	Multer + Cloudinary	Memory upload, MIME/size checks, signed storage, URLs saved in DB
Payments	Razorpay behind PaymentProvider interface	Order create, signature verification, webhooks, refunds
OTP/SMS	Twilio Verify or SMS API	Real OTP delivery, hashed OTP store, attempt limits
Maps	Google Maps / OSM	Geocoding, distance for delivery fee, route display
AI (optional)	FastAPI microservice	Maintenance prediction and pricing suggestions, called only when configured

3. Project Structure

3.1 Backend

```

server/
  src/
    app.js           express app, middleware, routes
    server.js        http server + socket bootstrap + db connect
    config/          env.js, db.js, cloudinary.js, razorpay.js, twilio.js
    middleware/      auth.js, rbac.js, validate.js, upload.js, error.js, rateLimit.js
    models/          User, Category, Equipment, Cart, Rental, Payment, Delivery,
                    DeliveryConditionReport, DeliveryEvidence, HandoverAcknowledgement,
                    Return, ReturnConditionReport, Dispute, MaintenanceRecord,
                    Review, Notification, KYC, TrackingLocation, Otp

    modules/
      auth/          routes, controller, service, validators
      users/ kyc/ categories/ equipment/ search/ cart/
      availability/ pricing/ rentals/ payments/
      deliveries/ tracking/ inspections/ handover/
      returns/ disputes/ maintenance/ reviews/ analytics/ notifications/
    sockets/         index.js (auth + rooms), tracking.js, notify.js
    services/providers/ payment/, sms/, kyc/, storage/, maps/, ai/
    jobs/            expireHolds.js, maintenanceDue.js, reminders.js
    utils/           ApiError, asyncHandler, dates(IST), money(paise), i18n
    scripts/         createIndexes.js, seedCategories.js (structure only)

```

3.2 Frontend

```

client/src/
  api/              axios.js, endpoints per module
  hooks/            useAuth, useSocket, useCart, useDebounce
  context/          AuthContext, SocketContext
  i18n/             en.json, hi.json, ta.json
  components/       ui/ (Button, Input, Modal, Table, Badge, Pagination, Toast)
                  layout/ (Navbar, Sidebar, Footer), equipment/, rental/, delivery/,
                  inspection/, tracking/ (map), dispute/

  pages/
    public/         Home, Marketplace, EquipmentDetail, Login, Register, ResetPassword
    customer/       Cart, Checkout, MyRentals, RentalDetail, Handover, ReturnRequest, Disputes
    owner/          Inventory, EquipmentForm, RentalRequests, Maintenance, Analytics
    transporter/    Assignments, DeliveryRun (GPS + checklist)
    inspector/      Inspections, ConditionReport
    admin/          Users, KYC, Categories, Disputes, Payments, Assignments

```

routes/	ProtectedRoute (role aware), router.jsx
styles/	tailwind theme tokens (white + red)

4. Data Model (LLD)

Conventions: money is stored as integer paise; all timestamps are UTC and rendered in IST; every collection has createdAt/updatedAt; references use ObjectId. Indexes are listed per model.

4.1 Entity relationships

User 1--1 KYC User 1--N Equipment (owner) Category 1--N Equipment
User 1--1 Cart Cart N--N Equipment (items)
Rental N--1 User(customer) ; N--1 User(owner) ; 1--N RentalItem(equipment, qty)
Rental 1--1 Payment(N for refunds) Rental 1--N Delivery (outbound + retries)
Delivery 1--1 DeliveryConditionReport Delivery 1--N DeliveryEvidence
Delivery 1--1 HandoverAcknowledgement Delivery 1--N TrackingLocation
Rental 1--1 Return ; Return 1--1 ReturnConditionReport ; Return 1--1 Delivery(type=RETURN)
Rental 0..N Dispute Equipment 1--N MaintenanceRecord Rental 0..1 Review
User 1--N Notification

User

Field	Type	Notes
name, email, phone	String	phone in E.164 (+91), unique; email unique lowercase
passwordHash	String	bcrypt cost 12; select:false
role	Enum	CUSTOMER OWNER TRANSPORTER INSPECTOR ADMIN
profile	Subdoc	role-specific: businessName, GSTIN (owner); vehicle, licence ref, serviceAreas (transporter); certifications (inspector)
addresses[]	Subdoc[]	line1, line2, landmark, city, state, pincode(6 digits), geo Point
language	Enum	en hi ta
isPhoneVerified, isEmailVerified, isActive	Boolean	
refreshTokens[]	Subdoc[]	hashed token, device, expiresAt (rotation)
passwordChangedAt	Date	invalidates older JWTs

Indexes: email(unique), phone(unique), role

Otp

Field	Type	Notes
userId / phone	Ref/String	
purpose	Enum	REGISTER LOGIN RESET_PASSWORD HANDOVER
codeHash	String	HMAC of OTP, never plain text
attempts	Number	max 5
expiresAt	Date	TTL index, 5 min
consumedAt	Date	

Indexes: expiresAt(TTL), phone+purpose

KYC

Field	Type	Notes
userId	Ref	unique
provider	String	DigiLocker / partner
providerRefId	String	verification reference
status	Enum	NOT_STARTED PENDING VERIFIED REJECTED EXPIRED
idType, idLast4	String	only masked data; raw Aadhaar never stored
documents[]	Subdoc[]	PAN/GST doc URL in private storage, if required
reviewedBy, reviewedAt, rejectionReason	Mixed	

Indexes: userId(unique), status

Category

Field	Type	Notes
slug	String	unique
names	Map	{en, hi, ta}
parent	Ref	optional nesting
keywords	Map<[String]>	per-language synonyms for search
icon, isActive, sortOrder	Mixed	created by admin only

Indexes: slug(unique), parent

Equipment

Field	Type	Notes
owner	Ref User	
category	Ref	
title, description	Map	{en, hi, ta}; en required
keywords	Map<[String]>	multilingual search terms
brand, model, specs	Mixed	specs key-value
images[]	Subdoc[]	url, publicId, isPrimary
totalQuantity	Number	units owned
pricing	Subdoc	dailyRate, weeklyRate, monthlyRate, securityDeposit (paise), minDays, maxDays
meterType	Enum	HOURS KM NONE
currentMeter	Number	updated from inspections
location	GeoJSON Point + city, state, pincode	pickup origin
deliveryOptions	Subdoc	pickupAllowed, deliveryAllowed, feePerKm, maxKm
status	Enum	DRAFT ACTIVE PAUSED ARCHIVED
ratingAvg, ratingCount	Number	maintained from Review

Indexes: text(title.*, keywords.*), category+status, owner, location(2dsphere), pricing.dailyRate

Cart

Field	Type	Notes
user	Ref	unique
items[]	Subdoc[]	equipment, quantity, startDate, endDate, deliveryMode, addressId
updatedAt	Date	stale carts pruned by job

Indexes: user(unique)

Rental

Field	Type	Notes
code	String	human-readable RH-YYMM-xxxxx
customer, owner	Ref User	
items[]	Subdoc[]	equipment, quantity, unit price snapshot, title snapshot
startDate, endDate	Date	half-open interval [start, end)
status	Enum	REQUESTED, CONFIRMED, PREPARING, IN_TRANSIT, ACTIVE, RETURN_PENDING, RETURNED, INSPECTING, COMPLETED, CANCELLED, DISPUTED
pricing	Subdoc	subtotal, discount, deliveryFee, platformFee, tax, deposit, total (paise) snapshot
deliveryAddress	Subdoc	snapshot
payment	Ref Payment	
statusHistory[]	Subdoc[]	status, at, by, note (audit trail)
cancellation	Subdoc	by, reason, at, refundAmount
holdExpiresAt	Date	auto-expire unpaid/unaccepted requests

Indexes: customer+status, owner+status, items.equipment+startDate+endDate+status, code(unique)

Payment

Field	Type	Notes
rental	Ref	
provider	String	razorpay
providerOrderId, providerPaymentId	String	unique sparse
amount, currency	Number/String	paise, INR
type	Enum	CHARGE DEPOSIT_HOLD REFUND DEPOSIT_RELEASE
status	Enum	CREATED AUTHORIZED CAPTURED FAILED REFUNDED PARTIAL_REFUND
signatureVerified	Boolean	from server-side HMAC check
webhookEvents[]	Subdoc[]	event id, at (idempotency)

Indexes: providerOrderId, rental

Delivery

Field	Type	Notes
rental	Ref	
type	Enum	OUTBOUND RETURN
transporter	Ref User	assigned by owner/admin
status	Enum	PENDING_ASSIGNMENT, ASSIGNED, ACCEPTED, PICKED_UP, IN_TRANSIT, ARRIVED, HANDED_OVER, FAILED, CANCELLED
pickup, drop	Subdoc	address snapshot + geo
scheduledWindow	Subdoc	from, to
distanceKm, fee	Number	from maps provider
failure	Subdoc	reason code, note, evidence, at, reattempt
timeline[]	Subdoc[]	status, at, geo

Indexes: rental, transporter+status

DeliveryConditionReport

Field	Type	Notes
delivery	Ref	unique
inspector / transporter	Ref User	who filled it
checklist[]	Subdoc[]	itemKey, label, state (OK MINOR MAJOR MISSING), note; template comes from Category
meterReading	Subdoc	value, unit, photo evidence ref
damages[]	Subdoc[]	component, description, severity, evidenceIds[]
missingComponents[]	Subdoc[]	component, qty
submittedAt	Date	immutable once submitted

Indexes: delivery(unique)

DeliveryEvidence

Field	Type	Notes
delivery, report	Ref	
kind	Enum	PHOTO VIDEO METER_PHOTO SIGNATURE
url, publicId	String	
capturedAt, geo	Date/Point	from device
uploadedBy	Ref User	
sha256	String	tamper check

Indexes: delivery, report

HandoverAcknowledgement

Field	Type	Notes
delivery	Ref	unique
customer	Ref	
method	Enum	OTP SIGNATURE
otpVerifiedAt / signature	Date/Evidence	real OTP sent to customer phone
acceptedReport	Boolean	
customerNotes, objections[]	Mixed	objections may open a dispute
acknowledgedAt	Date	

Indexes: delivery(unique)

Return

Field	Type	Notes
rental	Ref	unique
requestedAt, requestedBy	Mixed	
method	Enum	TRANSPORTER SELF_DROP OWNER_PICKUP
delivery	Ref Delivery	type=RETURN
status	Enum	REQUESTED, SCHEDULED, PICKED_UP, IN_TRANSIT, RECEIVED, INSPECTING, INSPECTED, CLOSED
conditionReport	Ref	
lateDays	Number	computed from endDate vs received

Indexes: rental(unique), status

ReturnConditionReport

Field	Type	Notes
return	Ref	unique
inspector	Ref User	
checklist[], meterReading, damages[], missingComponents[]	Same shape as delivery report	
comparison	Subdoc	generated diff vs DeliveryConditionReport: per item before/after/changed
evidenceIds[]	Ref[]	

Indexes: return(unique)

Dispute

Field	Type	Notes
rental, return?	Ref	
raisedBy	Ref User	customer or owner
category	Enum	DAMAGE MISSING_PARTS LATE_RETURN DEPOSIT OTHER
status	Enum	OPEN UNDER_REVIEW ACTION_REQUIRED RESOLVED CLOSED
claims[]	Subdoc[]	item, description, amountClaimed
evidence[]	Ref[]	
messages[]	Subdoc[]	sender, text, attachments, at
assignedAdmin	Ref User	
resolution	Subdoc	outcome, amounts, decidedBy, rationale, at (human decision only)

Indexes: rental, status, assignedAdmin

MaintenanceRecord

Field	Type	Notes
equipment	Ref	
type	Enum	SCHEDULED REPAIR INSPECTION CLEANING
quantityAffected	Number	units out of service
status	Enum	PLANNED IN_PROGRESS DONE CANCELLED
startDate, endDate	Date	blocks availability while active
sourceDispute / sourceReturn	Ref	optional link
cost, vendor, notes, meterAtService	Mixed	
attachments[]	Ref	

Indexes: equipment+startDate+endDate+status

Review

Field	Type	Notes
rental	Ref	unique per reviewer
reviewer, reviewee, equipment	Ref	
direction	Enum	CUSTOMER_TO_OWNER OWNER_TO_CUSTOMER
rating	Number	1-5
comment	String	
createdAt	Date	allowed only when rental.status = COMPLETED

Indexes: rental+reviewer(unique), equipment

Notification

Field	Type	Notes
user	Ref	
type	String	RENTAL_REQUESTED, DELIVERY_ASSIGNED, DISPUTE_UPDATED, ...
titleKey, params	String/Mixed	translated on the client (en/hi/ta)
link	String	
readAt	Date	
channels	Subdoc	inApp, sms, email delivery status

Indexes: user+readAt+createdAt

TrackingLocation

Field	Type	Notes
delivery	Ref	
transporter	Ref	
location	GeoJSON Point	from device GPS
accuracy, speed, heading	Number	
recordedAt	Date	device time
receivedAt	Date	server time

Indexes: delivery+recordedAt, location(2dsphere), TTL 90 days after delivery closes

5. API Design

Base path **/api/v1**. All responses use the envelope { success, data, meta?, error? }. Auth uses Authorization: Bearer <accessToken>; refresh token is an httpOnly cookie. Language via Accept-Language.

Module	Method + path	Access	Purpose
Auth	POST /auth/register, /auth/login, /auth/logout, /auth/refresh	Public / Auth	Account lifecycle and token rotation
Auth	POST /auth/otp/send, /auth/otp/verify	Public	Real OTP via SMS provider
Auth	POST /auth/forgot-password, /auth/reset-password, /auth/change-password	Public / Auth	Password recovery and change
Users	GET/PATCH /users/me, POST /users/me/addresses	Auth	Profile and addresses
KYC	POST /kyc/start, GET /kyc/status, POST /kyc/webhook, PATCH /admin/kyc/:id	Auth / Admin	Provider-driven verification and review
Categories	GET /categories, POST/PATCH/DELETE /categories	Public / Admin	Dynamic category tree
Equipment	GET /equipment (q, category, city, minPrice, maxPrice, from, to, sort, page, limit)	Public	Marketplace search with availability filter
Equipment	GET /equipment/:id, GET /equipment/:id/availability?from&to;&qty;	Public	Detail and live availability
Equipment	POST/PATCH/DELETE /equipment, POST /equipment/:id/images	Owner	Inventory CRUD and uploads
Cart	GET /cart, POST /cart/items, PATCH/DELETE /cart/items/:id	Customer	DB-backed cart
Pricing	POST /pricing/quote	Customer	Server-side price breakdown
Rentals	POST /rentals/checkout	Customer	Validate, price, hold, create payment order
Rentals	GET /rentals, GET /rentals/:id	Customer / Owner	Role-scoped dashboards
Rentals	PATCH /rentals/:id/accept, /reject, /cancel	Owner / Customer	Approval and cancellation
Payments	POST /payments/verify, POST /payments/webhook, POST /payments/:id/refund	Customer / Provider / Admin	Signature verification, refunds
Deliveries	POST /deliveries/:id/assign, PATCH /deliveries/:id/status	Owner, Admin / Transporter	Assignment and lifecycle
Deliveries	GET /deliveries/mine, GET /deliveries/:id	Transporter / parties	Dashboard and detail
Tracking	POST /deliveries/:id/location (also socket), GET /deliveries/:id/track	Transporter / parties	GPS ingest and history
Inspection	POST /deliveries/:id/condition-report, POST /deliveries/:id/evidence	Transporter, Inspector	Checklist, meter, photos, damages
Handover	POST /deliveries/:id/handover/otp, POST /deliveries/:id/handover/confirm	Transporter / Customer	OTP-verified acknowledgement
Delivery fail	POST /deliveries/:id/fail, POST /deliveries/:id/reattempt	Transporter / Owner, Admin	Failure workflow
Returns	POST /rentals/:id/return, PATCH /returns/:id/status	Customer / Transporter	Return workflow
Returns	POST /returns/:id/inspection, GET /returns/:id/comparison	Inspector	Return inspection and diff
Disputes	POST /disputes, GET /disputes, POST /disputes/:id/messages, PATCH /disputes/:id/status, POST /disputes/:id/resolve	Parties / Admin	Dispute handling
Maintenance	GET/POST/PATCH /equipment/:id/maintenance, GET /equipment/:id/passport	Owner / Admin	Records and Equipment Passport

Module	Method + path	Access	Purpose
Reviews	POST /reviews, GET /equipment/:id/reviews	Customer, Owner / Public	Post-completion reviews
Analytics	GET /analytics/owner, /analytics/equipment/:id, GET /pricing/suggestions/:equipmentId	Owner	Utilization, reliability, smart pricing
Notifications	GET /notifications, PATCH /notifications/:id/read, POST /notifications/read-all	Auth	Notification centre
Admin	GET /admin/users, PATCH /admin/users/:id, GET /admin/rentals, GET /admin/deliveries	Admin	Oversight

6. Core Logic (LLD)

6.1 Availability engine

An equipment unit count is never a flag; it is derived from overlapping reservations and maintenance blocks for the requested window.

```
blockingStatuses = [REQUESTED, CONFIRMED, PREPARING, IN_TRANSIT, ACTIVE,
                    RETURN_PENDING, RETURNED, INSPECTING, DISPUTED]
```

```
available(equipmentId, start, end) =
  equipment.totalQuantity
  - SUM(rental.item.quantity)           where rental.status in blockingStatuses
                                         and rental.start < end and rental.end > start
  - SUM(maintenance.quantityAffected) where status in [PLANNED, IN_PROGRESS]
                                         and maintenance.start < end and maintenance.end > start
```

Peak check: overlap is evaluated at every boundary date inside [start,end), because two shorter rentals may each overlap the window without overlapping each other.

- **Concurrency:** checkout runs in a MongoDB transaction: re-compute availability, insert Rental with status REQUESTED and holdExpiresAt, then commit. A second concurrent request sees the hold and fails with 409.
- **Holds:** a job (expireHolds) cancels REQUESTED rentals whose payment or owner decision is not completed by holdExpiresAt, releasing stock.
- **Buffer:** optional per-equipment turnaround days added to end date for blocking, so items in inspection are not double-booked.

6.2 Pricing calculation

```
days      = max(minDays, ceil((end - start) / 24h))
rate       = best of daily / weekly / monthly tier for 'days' (owner-defined)
subtotal   = SUM(unitCost(item, days) * quantity)
delivery   = distanceKm(owner -> customer) * feePerKm           (0 for pickup)
platform   = subtotal * platformFeePct                          (from settings collection)
tax         = (subtotal + delivery + platform) * gstPct          (configurable, not hardcoded)
deposit    = SUM(securityDeposit * quantity)                    (refundable)
total      = subtotal + delivery + platform + tax + deposit
```

Prices are always recalculated on the server; client-sent totals are ignored. The result is snapshotted into Rental.pricing so later rate changes never alter existing bookings.

6.3 Payment service architecture

- **PaymentProvider** interface: createOrder, verifySignature, capture, refund, parseWebhook. RazorpayProvider is the implementation.
- Flow: checkout creates Rental (REQUESTED) + Payment (CREATED) + Razorpay order; frontend opens Razorpay Checkout; backend verifies HMAC signature on /payments/verify and again on the webhook; webhook handler is idempotent via event id.
- Owner rejection or cancellation triggers provider refund according to the cancellation policy; deposit release happens after return inspection closes without an open dispute.
- If provider keys are missing the API responds 503 PAYMENT_NOT_CONFIGURED; no mock capture exists.

6.4 OTP and password flows

- OTP: 6 digits generated with `crypto.randomInt`, stored as HMAC (codeHash), 5-minute TTL, 5 attempts, resend cooldown and per-phone/IP rate limits; delivered by Twilio.
- Forgot password: single-use reset token (random 32 bytes, hashed in DB, 15-minute expiry) sent by email/SMS; reset invalidates all refresh tokens and sets `passwordChangedAt`.
- Login: bcrypt compare, short-lived access JWT (15 min), rotating refresh token (7 days) stored hashed.

6.5 KYC architecture

KycProvider interface (start, status, webhook). Aadhaar-based flows run through the provider or DigiLocker consent; RentalHub stores only providerRefId, status, ID type and last four digits. Owners and transporters must be VERIFIED before listing equipment or accepting deliveries; customers may require KYC above a configurable order value.

6.6 Multilingual search

- Equipment and Category carry titles and keyword arrays in en, hi, ta; MongoDB text index spans all language fields.
- Query pipeline: normalise (Unicode NFC, trim), optional transliteration lookup (for example a Hindi or Tamil word for 'tractor' resolves through category keywords), then \$text search plus category/keyword match, combined with filters.
- Ranking: text score, rating, and availability for requested dates. Pagination uses page + limit with total in meta.
- Keywords are entered by owners/admins in the equipment form, so no vocabulary is hardcoded in code.

6.7 Real-time layer (Socket.IO)

Room	Joined by	Events
user:{id}	Every logged-in user (JWT verified on handshake)	notification:new, rental:updated
rental:{id}	Customer, owner, assigned transporter, admin	rental:status, payment:updated
delivery:{id}	Parties to that delivery	delivery:status, tracking:location, handover:confirmed
dispute:{id}	Parties + assigned admin	dispute:message, dispute:status

Every state change follows one path: service updates MongoDB, writes a Notification document, then emits the socket event. The client React Query cache is invalidated by the event; if the socket is down, polling and the notification API provide the same data.

6.8 GPS tracking

- The transporter app uses the browser Geolocation API (`watchPosition`) and sends points over the socket; the server validates the sender is the assigned transporter and the delivery is IN_TRANSIT.
- Each point is persisted in TrackingLocation, then broadcast to delivery:{id}. Points are throttled (min distance/time) and ignored if accuracy is poor.
- If location permission is denied or GPS is unavailable the UI shows 'location unavailable'; no route is simulated.

6.9 Delivery and return inspection

- Checklist templates are stored per Category, so each equipment type gets its own real checklist items.
- A condition report is immutable after submission; corrections create an addendum record.
- Evidence uploads keep device timestamp, geo and sha256; meter reading must include a photo where meterType is not NONE.
- Handover requires an OTP sent to the customer's verified phone (or captured signature) and records whether the customer accepted the report; objections are stored and can open a dispute.

Comparison engine: ReturnConditionReport.comparison is computed by matching checklist itemKeys between delivery and return: for each item it stores before, after and changed. It also records meter difference, newly missing components and new damage entries. The output is descriptive only (for example 'item X changed from OK to MAJOR between delivery and return'). The system never states who caused the damage; blame is decided by an admin in the dispute resolution step.

6.10 Dispute workflow

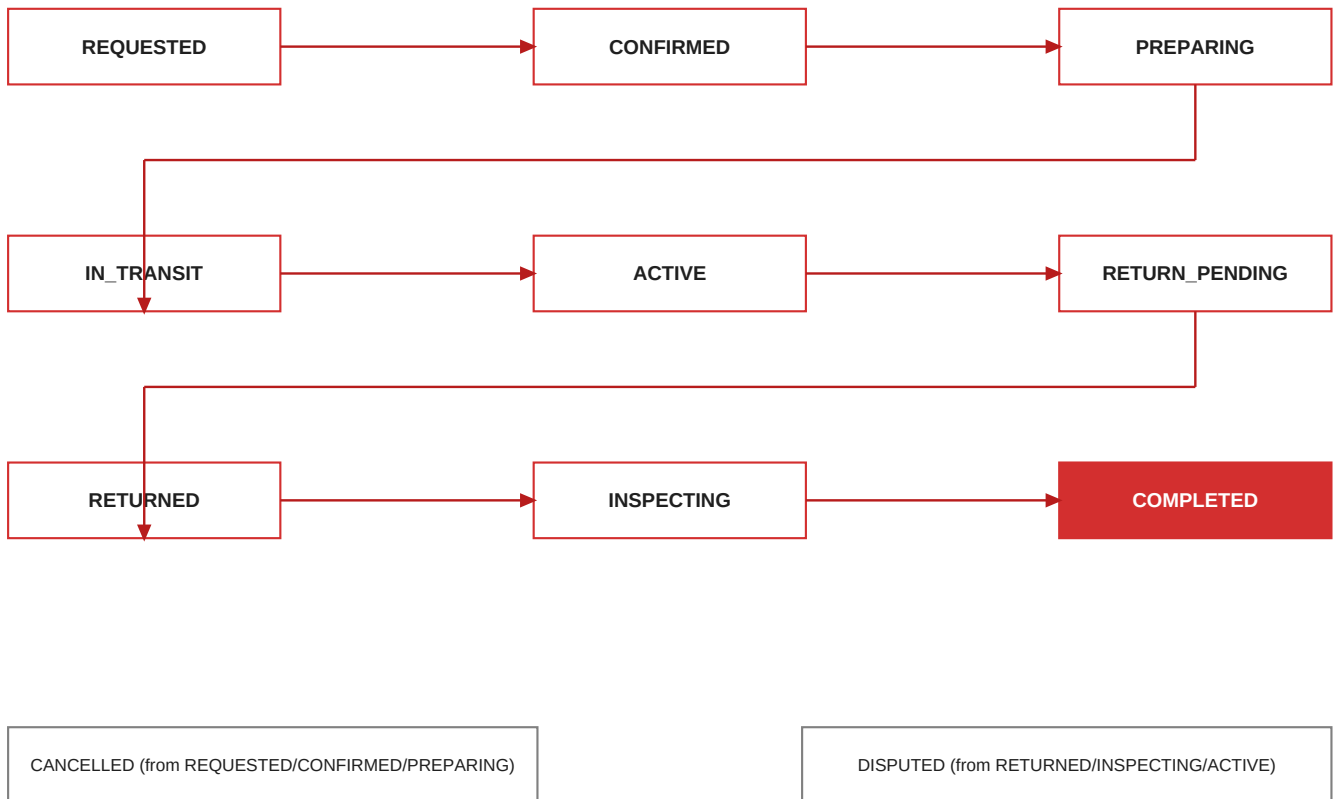
From	To	Actor	Rule
(new)	OPEN	Customer/Owner	Must attach rental, category, at least one claim; evidence optional but encouraged
OPEN	UNDER_REVIEW	Admin	Admin assigned; rental set to DISPUTED
UNDER_REVIEW	ACTION_REQUIRED	Admin	Requests evidence or response from a party; due date set
ACTION_REQUIRED	UNDER_REVIEW	Party	Responded with message/evidence
UNDER_REVIEW	RESOLVED	Admin	Resolution with rationale and amounts; deposit/refund actions executed
RESOLVED	CLOSED	System/Admin	After appeal window; rental moves to COMPLETED

6.11 Maintenance, prediction and analytics

- **Blocking:** active MaintenanceRecord rows reduce availability for their date range (see 6.1).
- **Equipment Passport:** read-only timeline aggregated from rentals, delivery/return reports, disputes and maintenance for one equipment, including meter history.
- **Prediction architecture:** MaintenancePredictor interface. Rule-based mode uses real meter/hours and service intervals set by the owner (due when current meter minus last service meter exceeds interval). Optional FastAPI mode receives real history and returns a risk score. If no data or no service is configured, the API returns 'insufficient data'.
- **Owner reliability:** computed from real rentals: acceptance rate, cancellation-by-owner rate, on-time delivery rate, dispute rate, average rating; shown only when a minimum number of rentals exists.
- **Utilization:** rented days divided by available days over a period per equipment (excluding maintenance days).
- **Smart pricing:** PricingAdvisor reads utilization, demand (search/cart counts), seasonality from past rentals and comparable listings, and returns a suggestion with the inputs used. Owner accepts or ignores; nothing changes automatically.

7. State Machines

7.1 Rental lifecycle



Transition	Trigger	Guard / side effects
REQUESTED to CONFIRMED	Owner accepts	Payment captured/authorized; availability re-checked; notify customer
REQUESTED to CANCELLED	Owner rejects, customer cancels, hold expires	Refund via provider; stock released
CONFIRMED to PREPARING	Owner starts preparation	Outbound Delivery created, awaiting transporter assignment
PREPARING to IN_TRANSIT	Transporter picks up	Delivery status PICKED_UP; GPS tracking starts
IN_TRANSIT to ACTIVE	Handover confirmed	Delivery condition report submitted + acknowledgement recorded
ACTIVE to RETURN_PENDING	Customer requests return or end date reached	Return record created
RETURN_PENDING to RETURNED	Equipment received	Return delivery completed or owner marks received
RETURNED to INSPECTING	Inspector starts	Return inspection opened
INSPECTING to COMPLETED	Inspection closed, no open dispute	Deposit released; review enabled; utilization updated
Any active to DISPUTED	Dispute opened	Deposit release frozen until resolution

Transitions are enforced in one `rentalStateMachine` map: allowed next states and permitted roles. Any other transition returns 409. Each change appends to `statusHistory`.

7.2 Delivery lifecycle

PENDING_ASSIGNMENT -> ASSIGNED -> ACCEPTED -> PICKED_UP -> IN_TRANSIT -> ARRIVED -> HANDED_OVER
 \-> FAILED -> (reattempt: ASSIGNED) | CANCELLED

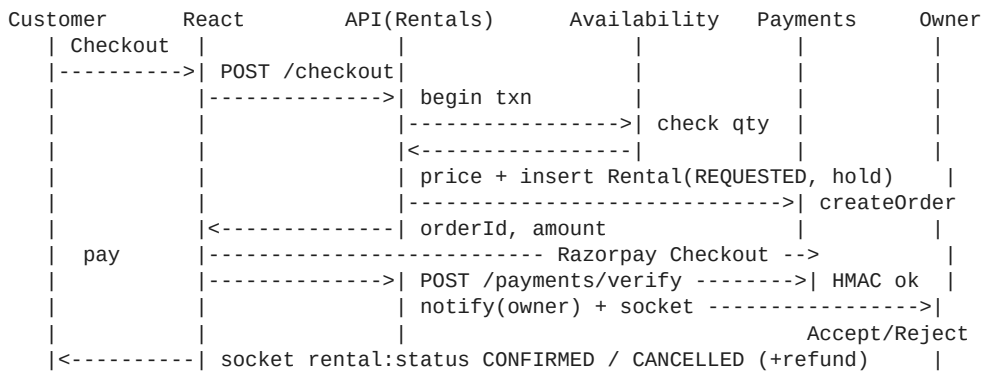
FAILED requires a reason code (customer unavailable, wrong address, refused, vehicle issue, other), a note and evidence; owner/admin then reattempts or cancels, and the customer is notified at every step.

7.3 Return lifecycle

REQUESTED -> SCHEDULED -> PICKED_UP -> IN_TRANSIT -> RECEIVED -> INSPECTING -> INSPECTED -> CLOSED

8. Key Sequence Flows

8.1 Checkout and booking



8.2 Delivery, GPS and handover

Owner/Admin -> assign transporter -> Transporter accepts -> pickup (status PICKED_UP)
 Transporter app --watchPosition--> socket tracking:location --> DB(TrackingLocation) --> room delivery:{id}
 Transporter at site: fills checklist + meter + photos -> POST condition-report
 API sends OTP to customer phone -> Customer enters OTP -> HandoverAcknowledgement saved
 Rental -> ACTIVE ; notifications to customer, owner, admin

8.3 Return, inspection and dispute

Customer requests return -> Return created -> return Delivery (transporter) -> RECEIVED
 Inspector submits ReturnConditionReport -> comparison generated vs delivery report
 No differences -> COMPLETED, deposit released, reviews enabled
 Differences -> party opens Dispute (evidence attached) -> admin review -> resolution -> CLOSED

9. Security and Access Control

Resource	Customer	Owner	Transporter	Inspector	Admin
Browse marketplace	Yes	Yes	Yes	Yes	Yes
Manage own equipment	No	Yes	No	No	Any
Cart and checkout	Yes	No	No	No	No
Accept/reject rental	No	Own rentals	No	No	Override
Assign transporter	No	Own rentals	No	No	Yes
Update delivery status/GPS	No	View	Assigned only	No	View
Delivery condition report	View	View	Create	Create	View
Return inspection	View	View	No	Create	View
Open dispute	Own rentals	Own rentals	No	No	Yes
Resolve dispute	No	No	No	No	Yes
Categories, KYC review, users	No	No	No	No	Yes

- Ownership checks on every :id route (a user can only read rentals, deliveries and reports they are party to).
- Password hashing bcrypt (cost 12), JWT secrets in env, refresh rotation, httpOnly + secure + sameSite cookies, helmet, CORS allow-list, rate limiting on auth/OTP.

- Input validation (zod) on every route; mongo-sanitize against operator injection; file uploads limited by MIME and size with server-side re-check.
- Payment webhooks verified with HMAC; private evidence URLs are signed and time-limited.
- Audit trail via statusHistory and admin action logs; PII minimised; no raw Aadhaar; logs never contain tokens, OTPs or full phone numbers.

10. Non-Functional Design

Area	Approach
Performance	Compound indexes above; lean() reads; pagination everywhere; React Query caching; lazy routes; image transformations via Cloudinary
Scalability	Stateless API; Socket.IO Redis adapter when running multiple instances; jobs runnable as a separate worker
Reliability	Transactions for booking/payment state; idempotent webhooks; retry with backoff on provider calls
Observability	Structured logging (pino), request IDs, health endpoint /health, error tracking hook
Localization	i18n JSON per language, server-translated notification keys, Intl formatting for INR and en-IN dates, +91 phone and 6-digit PIN validation
UI system	Tailwind theme tokens: primary #D32F2F, hover #B71C1C, surface #F7F7F7, border #E5E7EB, text #1F1F1F; no gradients or decorative effects; mobile-first responsive layouts
Config	All secrets and provider switches in environment variables; missing provider returns explicit 'not configured' errors
Testing	Jest + Supertest for services and APIs (availability overlap, state machine, pricing); mongodb-memory-server; React Testing Library for key flows
Deployment	Client on static hosting/CDN, server on container (Node 20), MongoDB Atlas replica set, Cloudinary, Razorpay/Twilio webhooks over HTTPS

11. Phased Implementation Plan

Phase	Backend deliverables	Frontend deliverables	Exit criteria
1	Auth, OTP, RBAC, users, KYC, categories, equipment CRUD + uploads, search, cart	Auth screens, marketplace, detail with date picker, cart, owner inventory, i18n shell	Owner lists equipment; customer finds it in en/hi/ta and adds to cart
2	Availability, pricing, checkout, payments, rental state machine, notifications, sockets	Checkout, customer and owner rental dashboards, notification centre	Overlapping bookings correctly rejected; payment verified; owner accepts in real time
3	Delivery, assignment, tracking, condition reports, evidence, handover, failure flow	Transporter dashboard/run screen, delivery views for all roles, report forms, live map	Real GPS points shown live; handover completed with OTP
4	Returns, return inspection + comparison, disputes, maintenance, passport, reviews, analytics	Return and dispute UIs, maintenance and passport, reviews, owner analytics	Full lifecycle from listing to review passes end to end

Build order inside each phase: models and indexes, services with tests, routes, then UI. Each phase leaves the app deployable, and every later phase extends (never bypasses) the earlier services.